EXPERIMENT 6

Name-Spandan Deb
Class-D20A
Roll no-20

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:
What is a smart contract?
A smart contract is a self-executing digital agreement stored on a blockchain that automatically enforces its terms when predefined conditions are met, eliminating the need for intermediaries.

Significance of smart Contracts in Ethereum Blockchain

1. Programmability – Ethereum smart contracts make the blockchain programmable, enabling complex logic beyond simple payments. Decentralized Applications (dApps) – They power dApps like Uniswap, Aave, and OpenSea, running without central servers.
2. Trustless Automation – Transactions execute automatically without intermediaries, ensuring fairness and transparency.
3. Foundation for DeFi & NFTs – Smart contracts enable decentralized finance, NFT ownership transfers, and DAO governance.
4. Global Accessibility – Anyone can interact with Ethereum smart contracts worldwide, creating a borderless digital economy.

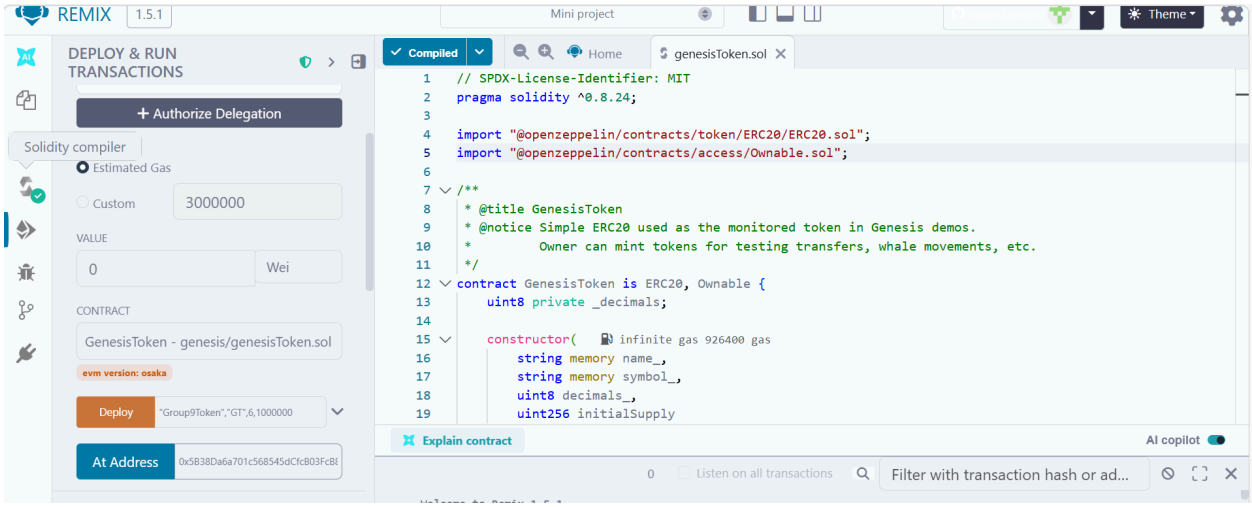
Code:
GenesisToken.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";
```

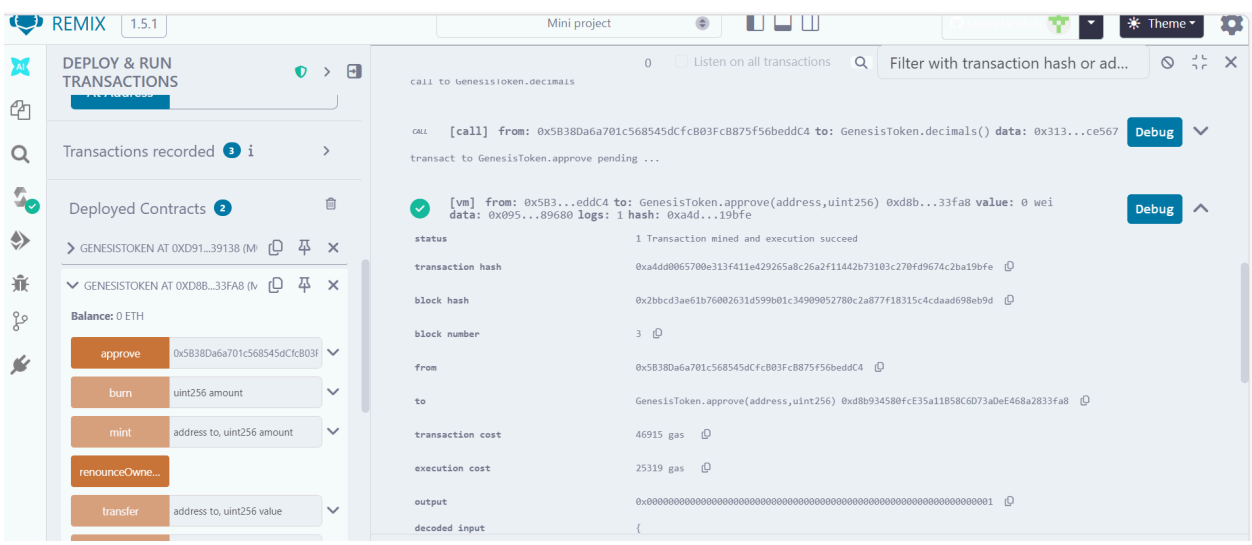
```
/**
 * @title GenesisToken
 * @notice Simple ERC20 used as the monitored token in Genesis demos.
 *         Owner can mint tokens for testing transfers, whale movements, etc.
 */
contract GenesisToken is ERC20, Ownable {
    uint8 private _decimals;

    constructor(
        string memory name_,
        string memory symbol_,
        uint8 decimals_,
        uint256 initialSupply
    ) ERC20(name_, symbol_) Ownable(msg.sender) {
        _decimals = decimals_;
        _mint(msg.sender, initialSupply * (10 ** decimals_));
    }

    function decimals() public view override returns (uint8) {
        return _decimals;
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    function burn(uint256 amount) external {
        _burn(msg.sender, amount);
    }
}
```

Output-

Compiled and deployed the contract by providing token name,token symbol,decimals and initial supply.

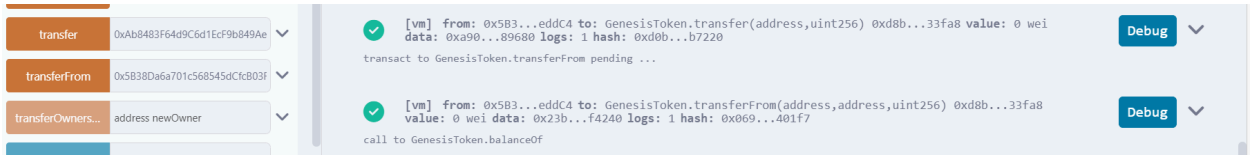


Approving the spender by providing their address and the amount the token holder wants to distribute .



Burning and minting tokens

Transfering tokens from your own balance using transfer and transfer tokens from someone else's balance to another address only if the address is approved using transfer from.



Checking the allowance of a wallet address after it has been approved and checking the balance of the owner.

Code-
GenesisVault.sol

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
```

```
import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

/**
 * @title GenesisVault
 * @notice On-chain vault that emits rich events for every deposit, withdrawal,
 *         and internal transfer.  Genesis monitors these events in real time.
 *
 *         KEY BLOCKCHAIN FEATURES SHOWCASED:
 *           1. Event-driven architecture (Solidity `event`)
 *           2. Inbound/outbound tracking on-chain
 *           3. Reentrancy protection
 *           4. Per-user balance accounting
 *           5. Emergency pause (circuit-breaker pattern)
 */
contract GenesisVault is Ownable, ReentrancyGuard {
    using SafeERC20 for IERC20;

    // ═══════════════════════════════ State ═══════════════════════════════
    IERC20 public immutable token;

    mapping(address => uint256) public balances;
    uint256 public totalDeposits;
    uint256 public totalWithdrawals;
    bool public paused;

    // ═══════════════════════════════ Events (the core of Genesis monitoring)
═══════════

    /// @notice Emitted on every deposit — Genesis indexes this immediately.
    event Deposit(
        address indexed user,
        uint256 amount,
        uint256 newBalance,
        uint256 timestamp
    );
```

```
/// @notice Emitted on every withdrawal — potential "outbound whale" signal.
event Withdrawal(
    address indexed user,
    uint256 amount,
    uint256 remainingBalance,
    uint256 timestamp
);

/// @notice Vault-internal transfer between two users.
event InternalTransfer(
    address indexed from,
    address indexed to,
    uint256 amount,
    uint256 timestamp
);

/// @notice Large movement detected on-chain — threshold exceeded.
event LargeMovement(
    address indexed user,
    string movementType,  // "deposit" | "withdrawal"
    uint256 amount,
    uint256 thresholdUsed,
    uint256 timestamp
);

/// @notice Emergency pause toggled.
event EmergencyAction(
    string action,   // "paused" | "unpaused"
    address indexed triggeredBy,
    uint256 timestamp
);

// ──────────────────────────────── Modifiers ────────────────────────────────
modifier whenNotPaused() {
```

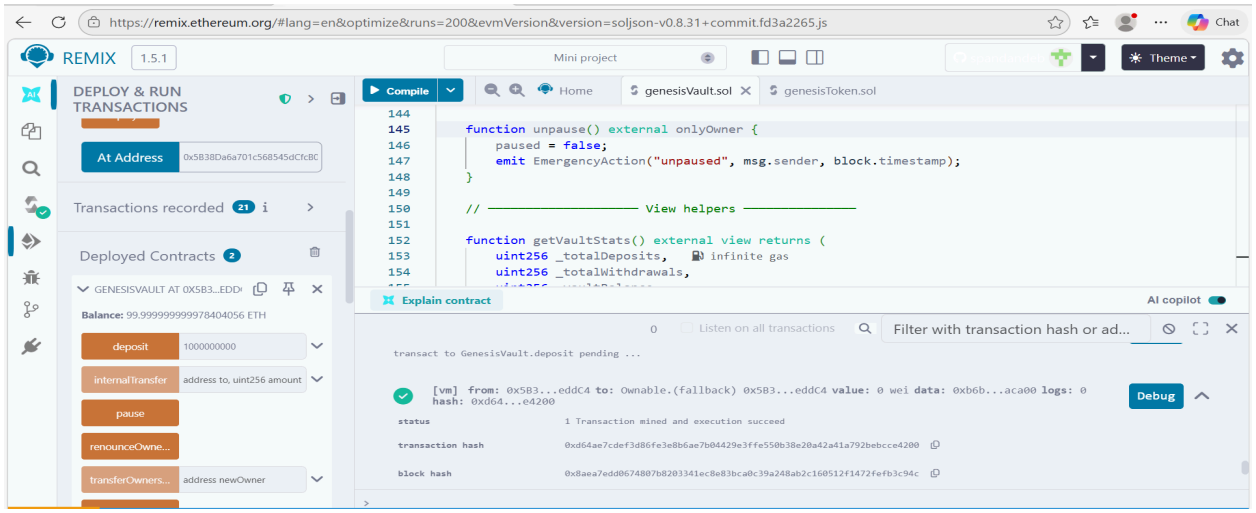
```
    require(!paused, "Vault: paused");
    _;
}

// ──────────────────────────────── Constructor ──────────────────────────
constructor(address tokenAddress) Ownable(msg.sender) {
    token = IERC20(tokenAddress);
}

// ──────────────────────────────── Core functions ────────────────────

/**
 * @notice Deposit tokens into the vault.
 * @param amount Number of token units (with decimals).
 */
function deposit(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");

    token.safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender] += amount;
    totalDeposits += amount;

    emit Deposit(msg.sender, amount, balances[msg.sender], block.timestamp);

    // On-chain large-movement detection (threshold hard-coded for demo;
    // the ThresholdEngine contract provides user-customisable thresholds).
    if (amount >= 100_000 * 1e6) {
        emit LargeMovement(msg.sender, "deposit", amount, 100_000 * 1e6,
block.timestamp);
    }
}

/**
 * @notice Withdraw tokens from the vault.
 */
```

```
    function withdraw(uint256 amount) external nonReentrant whenNotPaused {
        require(amount > 0, "Vault: zero amount");
        require(balances[msg.sender] >= amount, "Vault: insufficient balance");

        balances[msg.sender] -= amount;
        totalWithdrawals += amount;
        token.safeTransfer(msg.sender, amount);

        emit Withdrawal(msg.sender, amount, balances[msg.sender],
block.timestamp);

        if (amount >= 100_000 * 1e6) {
            emit LargeMovement(msg.sender, "withdrawal", amount, 100_000 * 1e6,
block.timestamp);
        }
    }

    /**
     * @notice Transfer balance to another vault user without touching the token
contract.
     *        Demonstrates internal accounting — useful for Layer-2 patterns.
     */
    function internalTransfer(address to, uint256 amount) external nonReentrant
whenNotPaused {
        require(to != address(0) && to != msg.sender, "Vault: invalid recipient");
        require(balances[msg.sender] >= amount, "Vault: insufficient balance");

        balances[msg.sender] -= amount;
        balances[to] += amount;

        emit InternalTransfer(msg.sender, to, amount, block.timestamp);
    }

    // ──────────────────────────────── Admin / Emergency ────────────────
```

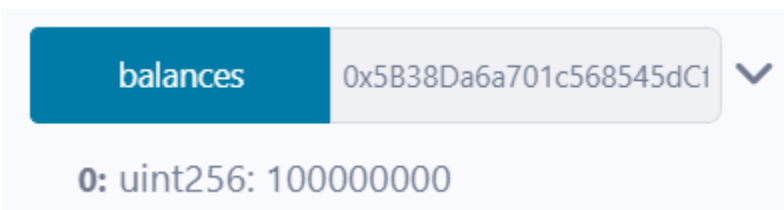
```
    function pause() external onlyOwner {
        paused = true;
        emit EmergencyAction("paused", msg.sender, block.timestamp);
    }

    function unpause() external onlyOwner {
        paused = false;
        emit EmergencyAction("unpaused", msg.sender, block.timestamp);
    }

    // ──────────────────────────────── View helpers ────────────────────────

    function getVaultStats() external view returns (
        uint256 _totalDeposits,
        uint256 _totalWithdrawals,
        uint256 _vaultBalance,
        bool _paused
    ) {
        return (totalDeposits, totalWithdrawals, token.balanceOf(address(this)),
paused);
    }
}
```

Output-

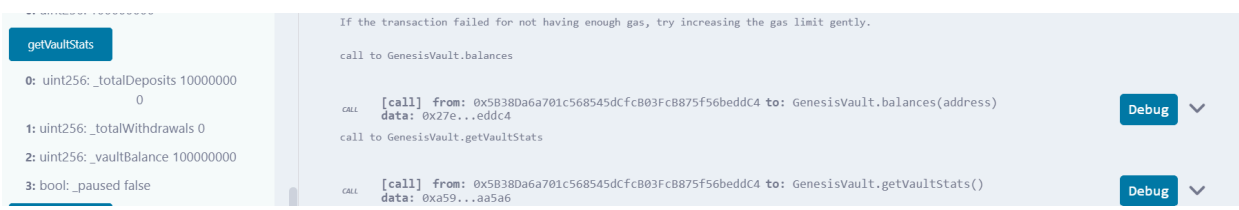
After minting new tokens,and approving we are depositing an amount to the vault



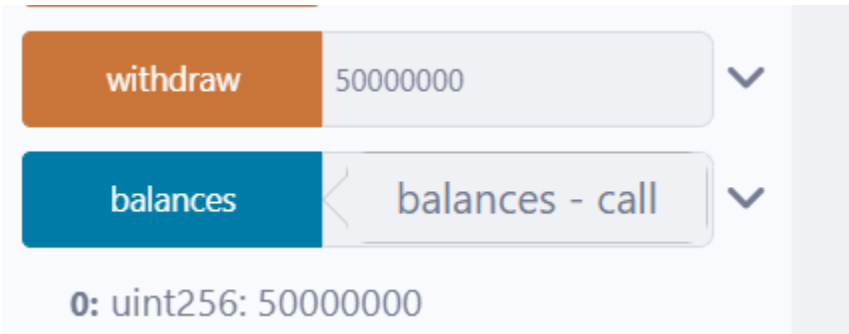
Balance is the same as the amount deposited.



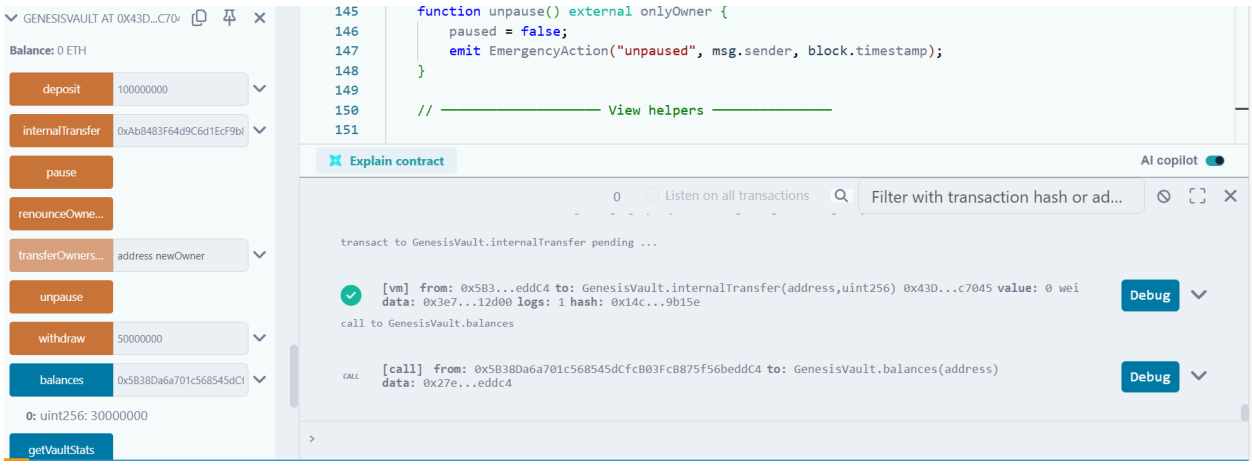
The stats of the Vault



Withdrew 50 tokens and checked balance to confirm .



Transferred 20 tokens to a different account via internal transfer and checked the balance which reduced from 50 tokens to 30 tokens.



Checked the balance of the account to which the tokens were transferred

Conclusion-

In this experiment, we successfully deployed and executed the GenesisToken and GenesisVault smart contracts using Remix IDE. We learned how to transfer tokens securely between contracts and performed deposit, withdrawal, and internal transfer operations.